

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

ОТЧЕТ
по лабораторной работе № 1
по дисциплине «Методы машинного обучения»
на тему
«Основы Python»
Вариант 17

Выполнил: студент гр. 351002
Проверил:

Лашенко Е.Н.
Лебедевич А.В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью работы является закрепление средств языка Python для обработки чисел, текста и коллекций, применения функций с обработкой исключений, а также построения классов, наследования и методов разных видов. Индивидуальное задание выполнено по варианту 17.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Для набора однотипных значений в языке Python применяют четыре коллекции:

- список сохраняет порядок и допускает повторы;
- кортеж после создания не изменяется;
- множество хранит только уникальные элементы;
- словарь сопоставляет ключ и значение.

Ключ словаря должен быть неизменяемым, поэтому число и строка для ключа подходят, а список – нет. Тип значения заранее не объявляют: он определяется при выполнении.

Простым называют натуральное число, у которого ровно два различных делителя: единица и само число. Единица простым числом не является. Делитель, больший квадратного корня из числа, всегда сопровождается меньшим делителем, поэтому перебор достаточно вести от двух до целой части квадратного корня.

Текст просматривают посимвольно. Букву относят к гласным или согласным, если она входит в заранее составленный набор. Перед сравнением регистр приводят к одному, иначе заглавная и строчная буква будут сочтены разными. Словом считают непустой фрагмент между пробелами. Разделение строки по пробелам отбрасывает пустые фрагменты и даёт число слов.

Если значение встречается в списке k раз, любые два вхождения образуют одну пару. Число пар для одного значения определяют по формуле

$$C = \frac{k(k-1)}{2}, \quad (2.1)$$

где C – число пар;

k – число вхождений значения.

Общее число пар равно сумме величин C по всем различным значениям. При k , равном нулю или единице, пар нет.

Частоту цифр накапливают словарём. Ключом делают целое число, значением – сколько раз эта цифра встретилась. Если ключа ещё нет, счётчик начинают с нуля и затем увеличивают.

Функция собирает действия, которые нужны больше одного раза, и возвращает результат. Один и тот же вызов может принять аргумент разного типа, если внутри выполнить проверку типа. Логическое значение в Python построено как частный случай целого числа, поэтому проверку логического типа ставят раньше проверки целого, иначе ветвь числа перехватит и истину, и ложь.

Ошибку во время выполнения перехватывают конструкцией `try/except`, и программа не останавливается целиком. В `except` указывают вид ошибки: отдельная ветвь для деления на ноль, отдельная – для неверного ввода. Блок `finally` выполняется и после успешного расчёта, и после перехвата ошибки. В него помещают завершающее действие, которое нельзя пропустить.

Класс задаёт поля и операции объекта. Атрибут, записанный в теле класса, общий для всех объектов. Атрибут, созданный в инициализаторе, принадлежит одному объекту. Метод экземпляра получает ссылку на объект и может читать оба вида атрибутов, а также глобальные имена модуля, если локальная переменная их не закрывает.

Наследование передаёт дочернему классу поля и методы родителя. Метод с тем же именем в потомке заменяет родительский только у объектов потомка. Объект базового класса по-прежнему вызывает исходную версию.

Помимо метода экземпляра используют два особых вида. Статический метод не получает ни объект, ни класс и работает как обычная функция внутри пространства имён класса. Метод класса получает сам класс. Им создают объект другим способом, например разбирают строку и вызывают инициализатор, либо читают общий счётчик объектов.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Задание выполнено по варианту 17 и состоит из трёх частей.

3.1 Часть 1

1 Вывести на экран все простые числа в заданном диапазоне. Диапазон вводится с клавиатуры.

2 С клавиатуры вводится текст. Определить, сколько в нём гласных и сколько согласных. При равенстве этих количеств вывести все гласные буквы. Посчитать количество слов в тексте.

3 Дан список чисел. Посчитать, сколько в нём пар элементов, равных друг другу. Любые два элемента, равные друг другу, образуют одну пару.

4 Дана строка в виде случайной последовательности чисел от 0 до 9. Создать словарь: ключ – само число типа `int`, значение – сколько раз оно встретилось.

5 Реализовать программу «Магазин автозапчастей» с шестью пунктами меню. В словаре ключ – название продукции, значение – список из описания, цены и количества на складе. Пункты меню: просмотр описания, просмотр цены, просмотр количества, вся информация, покупка, выход. При покупке с клавиатуры вводят название и количество, символ `n` завершает покупку. Нужно посчитать стоимость выбранных товаров и остаток в исходном списке.

6 Дан кортеж целых чисел. Найти индекс первого отрицательного элемента.

3.2 Часть 2

Все задания части 2 выполнены функциями и конструкцией `try/except/finally`.

1 Написать функцию для вычисления факториала числа.

2 Написать функцию с одним аргументом. Для списка найти сумму отрицательных чисел и удалить повторяющиеся элементы. Для множества посчитать количество слов. Для числа определить, простое оно или нет. Для строки найти количество гласных и согласных и определить самое длинное слово. Проверку выполнить для всех перечисленных случаев.

3 Проверить, все ли строки матрицы содержат хотя бы один положительный элемент. Если да, изменить знаки всех элементов матрицы на обратные.

4 Написать программу, которая демонстрирует работу try/except/finally.

3.3 Часть 3

1 Класс Example содержит три метода, две статические переменные и две динамические. Первый метод создаёт переменную и выводит её. Вторым возвращает сумму двух глобальных переменных. Третий возвращает результат возведения первой динамической переменной во вторую. Создать объект, напечатать методы и переменную a.

2 Класс Country хранит столицу, площадь и численность населения. Создать список объектов и вывести страны по заданной площади и по заданной численности населения.

3 Класс Stationery задаёт атрибут title и метод draw с сообщением о запуске отрисовки. Дочерние классы Pen, Pencil и Handle переопределяют draw и выводят собственное сообщение. Создать экземпляры и проверить метод для каждого.

4 Придумать собственный класс и реализовать в нём метод экземпляра, статический метод и метод класса.

4 РЕШЕНИЕ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Программы оформлены в трёх блокнотах: part1.ipynb, part2.ipynb и part3.ipynb. Ниже приведён код каждого задания и результат контрольного запуска.

4.1 Часть 1. Базовые конструкции

4.1.1 Простые числа в диапазоне

Число проверяют перебором делителей от двух до целой части квадратного корня. Если делитель найден, число составное. Границы диапазона читают с клавиатуры. При обратном порядке границ их меняют местами.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

a = int(input("Введите начало диапазона: "))
b = int(input("Введите конец диапазона: "))

if a > b:
    a, b = b, a

primes = [x for x in range(a, b + 1) if is_prime(x)]
print("Простые числа в диапазоне:", primes if primes else "нет")
```

Для диапазона от 10 до 30 получены числа 11, 13, 17, 19, 23 и 29. Результат запуска приведён на рисунке 4.1.

```
Введите начало диапазона: 10
Введите конец диапазона: 30
Простые числа в диапазоне: [11, 13, 17, 19, 23, 29]
```

Рисунок 4.1 – Простые числа в диапазоне от 10 до 30

4.1.2 Гласные, согласные и число слов

Буквы сравнивают с наборами гласных и согласных русского и латинского алфавитов. При равенстве количеств собранные гласные печатают в

том порядке, в каком они стояли в тексте. Число слов равно числу фрагментов после разделения по пробелам.

```
vowels_set = set("aeёиоуыэюяaeiouy")
consonants_set = set("бвгджзйклмнпрстфхцчщшбсcdfghjklmnpqrstvwxyz")

text = input("Введите текст: ")

vowels_count = 0
consonants_count = 0
found_vowels = []

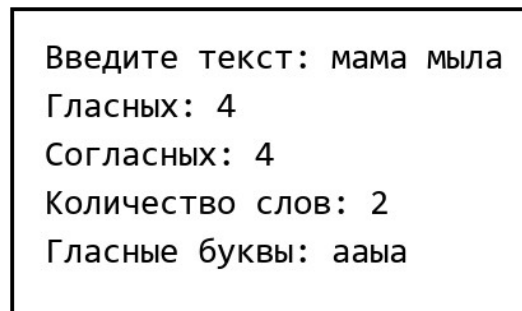
for ch in text.lower():
    if ch in vowels_set:
        vowels_count += 1
        found_vowels.append(ch)
    elif ch in consonants_set:
        consonants_count += 1

words = text.split()
words_count = len(words)

print(f"Гласных: {vowels_count}")
print(f"Согласных: {consonants_count}")
print(f"Количество слов: {words_count}")

if vowels_count == consonants_count:
    print("Гласные буквы:", "".join(found_vowels))
```

В тексте «мама мыла» гласных и согласных по четыре, поэтому напечатаны гласные. Слов в тексте два. Результат приведён на рисунке 4.2.



```
Введите текст: мама мыла
Гласных: 4
Согласных: 4
Количество слов: 2
Гласные буквы: ааыа
```

Рисунок 4.2 – Подсчёт гласных, согласных и слов

4.1.3 Пары равных элементов

Для каждого значения число пар считают по формуле (2.1) и суммируют. В контрольном списке единица и двойка встречаются по три раза, четвёрка – два раза, тройка и пятёрка – по одному разу.

```
from collections import Counter
```

```

numbers = [1, 2, 3, 2, 1, 1, 4, 5, 4, 2]
print("Список:", numbers)

pairs = 0
for count in Counter(numbers).values():
    pairs += count * (count - 1) // 2

print("Количество пар равных элементов:", pairs)

```

Число пар равно семи: три пары единиц, три пары двоек и одна пара четвёрок. Результат приведён на рисунке 4.3.

Список: [1, 2, 3, 2, 1, 1, 4, 5, 4, 2] Количество пар равных элементов: 7
--

Рисунок 4.3 – Число пар равных элементов

4.1.4 Словарь частот цифр

Строку из двадцати цифр собирают генератором случайных чисел от 0 до 9. При обходе строки символ переводят в `int` и увеличивают счётчик соответствующего ключа.

```

import random

sequence = "".join(str(random.randint(0, 9)) for _ in range(20))
print("Последовательность:", sequence)

digit_count = {}
for ch in sequence:
    key = int(ch)
    digit_count[key] = digit_count.get(key, 0) + 1

print("Словарь:", digit_count)

```

Состав строки при каждом запуске другой. На рисунке 4.4 показан один запуск: ключи словаря – целые числа, значения – количества вхождений.

Последовательность: 86454284103664856280 Словарь: {8: 4, 6: 4, 4: 4, 5: 2, 2: 2, 1: 1, 0: 2, 3: 1}

Рисунок 4.4 – Словарь частот цифр для одной случайной строки

4.1.5 Магазин автозапчастей

Карточка товара – список из трёх полей: описание, цена в рублях, остаток в штуках. Пункты 1–4 только печатают сведения и склад не меняют. В пункте 5 название и количество читают в цикле до ввода n. Покупку проводят, если товар есть и запрошенное количество не больше остатка. Стоимость позиции прибавляют к итогу, остаток уменьшают.

```
shop = {
    "Фильтр масляный": ["Очищает масло двигателя", 850, 15],
    "Свеча зажигания": ["Воспламенение топливной смеси", 320, 40],
    "Тормозная колодка": ["Комплект передних колодок", 2100, 8],
    "Аккумулятор": ["12В, 60 А·ч", 5500, 5],
    "Ремень ГРМ": ["Ремень газораспределительного механизма", 1800, 12],
}

def show_description():
    print("\n--- Описание ---")
    for name, info in shop.items():
        print(f"{name} – {info[0]}")

def show_price():
    print("\n--- Цены ---")
    for name, info in shop.items():
        print(f"{name} – {info[1]} руб.")

def show_quantity():
    print("\n--- Количество ---")
    for name, info in shop.items():
        print(f"{name} – {info[2]} шт.")

def show_all():
    print("\n--- Вся информация ---")
    for name, info in shop.items():
        print(f"{name}: описание – {info[0]}; цена – {info[1]} руб.; количество – {info[2]} шт.")

def buy():
    total = 0
    print("\n--- Покупка (n – выход) ---")
    while True:
        name = input("Название продукции: ").strip()
        if name.lower() == "n":
            break
        if name not in shop:
            print("Такого товара нет.")
            continue
        try:
            qty = int(input("Количество: "))
        except ValueError:
            print("Введите целое число.")
            continue
        if qty <= 0:
```

```

        print("Количество должно быть больше 0.")
        continue
    if qty > shop[name][2]:
        print(f"Недостаточно товара. В наличии: {shop[name][2]} шт.")
        continue
    cost = shop[name][1] * qty
    shop[name][2] -= qty
    total += cost
    print(f"Куплено: {name} × {qty} = {cost} руб. Осталось:
{shop[name][2]} шт.")

    print(f"\nИтого к оплате: {total} руб.")
    print("Остатки на складе:")
    for name, info in shop.items():
        print(f"    {name}: {info[2]} шт.")

while True:
    print("\n===== Магазин автозапчастей =====")
    print("1. Просмотр описания")
    print("2. Просмотр цены")
    print("3. Просмотр количества")
    print("4. Вся информация")
    print("5. Покупка")
    print("6. До свидания")

    choice = input("Выберите пункт меню: ").strip()

    if choice == "1":
        show_description()
    elif choice == "2":
        show_price()
    elif choice == "3":
        show_quantity()
    elif choice == "4":
        show_all()
    elif choice == "5":
        buy()
    elif choice == "6":
        print("До свидания!")
        break
    else:
        print("Неверный пункт меню.")

```

В первом запуске выбраны пункты 1 и 4: выведены описание и полная карточка товаров. Этот просмотр показан на рисунке 4.5.

===== Магазин автозапчастей =====

1. Просмотр описания
 2. Просмотр цены
 3. Просмотр количества
 4. Вся информация
 5. Покупка
 6. До свидания
- Выберите пункт меню: 1

--- Описание ---

Фильтр масляный – Очищает масло двигателя
Свеча зажигания – Воспламенение топливной смеси
Тормозная колодка – Комплект передних колодок
Аккумулятор – 12В, 60 А·ч
Ремень ГРМ – Ремень газораспределительного механизма

===== Магазин автозапчастей =====

1. Просмотр описания
 2. Просмотр цены
 3. Просмотр количества
 4. Вся информация
 5. Покупка
 6. До свидания
- Выберите пункт меню: 4

--- Вся информация ---

Фильтр масляный: описание – Очищает масло двигателя; цена – 850 руб.; количество – 15 шт.
Свеча зажигания: описание – Воспламенение топливной смеси; цена – 320 руб.; количество – 40 шт.
Тормозная колодка: описание – Комплект передних колодок; цена – 2100 руб.; количество – 8 шт.
Аккумулятор: описание – 12В, 60 А·ч; цена – 5500 руб.; количество – 5 шт.
Ремень ГРМ: описание – Ремень газораспределительного механизма; цена – 1800 руб.; количество – 12 шт.

===== Магазин автозапчастей =====

1. Просмотр описания
 2. Просмотр цены
 3. Просмотр количества
 4. Вся информация
 5. Покупка
 6. До свидания
- Выберите пункт меню: 6
До свидания!

Рисунок 4.5 – Просмотр описания и полной карточки товаров

Во втором запуске куплены два масляных фильтра по 850 руб. и три свечи по 320 руб. Сумма равна 2 660 руб. Остаток фильтров стал 13 шт., свечей – 37 шт. Остальные позиции не изменились. Завершение работы выполнено пунктом 6. Покупка показана на рисунке 4.6.

```

===== Магазин автозапчастей =====
1. Просмотр описания
2. Просмотр цены
3. Просмотр количества
4. Вся информация
5. Покупка
6. До свидания
Выберите пункт меню: 5

--- Покупка (n – выход) ---
Название продукции: Фильтр масляный
Количество: 2
Куплено: Фильтр масляный × 2 = 1700 руб. Осталось: 13 шт.
Название продукции: Свеча зажигания
Количество: 3
Куплено: Свеча зажигания × 3 = 960 руб. Осталось: 37 шт.
Название продукции: n

Итого к оплате: 2660 руб.
Остатки на складе:
    Фильтр масляный: 13 шт.
    Свеча зажигания: 37 шт.
    Тормозная колодка: 8 шт.
    Аккумулятор: 5 шт.
    Ремень ГРМ: 12 шт.

===== Магазин автозапчастей =====
1. Просмотр описания
2. Просмотр цены
3. Просмотр количества
4. Вся информация
5. Покупка
6. До свидания
Выберите пункт меню: 6
До свидания!

```

Рисунок 4.6 – Покупка товаров и остатки на складе

4.1.6 Индекс первого отрицательного элемента

Кортеж просматривают слева направо. На первом отрицательном значении цикл останавливают и сохраняют индекс. Если такого элемента нет, об этом печатают отдельное сообщение.

```

numbers = (5, 3, 0, 8, -2, 4, -7, 1)
print("Кортеж:", numbers)

```

```

index = None
for i, value in enumerate(numbers):
    if value < 0:
        index = i
        break

if index is not None:
    print(f"Индекс первого отрицательного элемента: {index} (значение {numbers[index]})")
else:
    print("Отрицательных элементов нет.")

```

В кортеже (5, 3, 0, 8, -2, 4, -7, 1) первый отрицательный элемент равен минус 2 и имеет индекс 4. Нумерация начинается с нуля. Результат приведён на рисунке 4.7.

Кортеж: (5, 3, 0, 8, -2, 4, -7, 1) Индекс первого отрицательного элемента: 4 (значение -2)
--

Рисунок 4.7 – Индекс первого отрицательного элемента кортежа

4.2 Часть 2. Функции и исключения

4.2.1 Факториал

Аргумент приводят к целому. Для отрицательного числа формируют `ValueError`. Произведение накапливают циклом от двух до `n`. Ноль и единица дают факториал, равный единице. Сообщение о завершении стоит в `finally`, поэтому печатается и при успехе, и при ошибке.

```

def factorial(n):
    """Вычисляет факториал числа n."""
    try:
        n = int(n)
        if n < 0:
            raise ValueError("Факториал определён только для неотрицательных чисел")
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result
    except (TypeError, ValueError) as e:
        print(f"Ошибка: {e}")
        return None
    finally:
        print("Вычисление факториала завершено.")

print("5! =", factorial(5))
print("0! =", factorial(0))

```

```
print("(-3)! =", factorial(-3))
print("'abc'! =", factorial("abc"))
```

Факториал 5 равен 120, факториал 0 равен 1. Для минус 3 и для строки «abc» выведено сообщение об ошибке, значение не получено. Блок `finally` выполнен во всех четырёх вызовах. Его сообщение появляется раньше строки с результатом, потому что печать происходит внутри функции, до возврата значения. Запуск показан на рисунке 4.8.

```
Вычисление факториала завершено.
5! = 120
Вычисление факториала завершено.
0! = 1
Ошибка: Факториал определён только для неотрицательных чисел
Вычисление факториала завершено.
(-3)! = None
Ошибка: invalid literal for int() with base 10: 'abc'
Вычисление факториала завершено.
'abc'! = None
```

Рисунок 4.8 – Вычисление факториала и обработка неверного аргумента

4.2.2 Обработка аргумента по его типу

Ветвь выбирают по типу аргумента. У списка суммируют отрицательные числа и удаляют повторы с сохранением порядка. У множества считают непустые строки. Число проверяют на простоту. У строки считают гласные и согласные и выбирают самое длинное слово. Неподдерживаемый тип перехватывают в `except`. Блок `finally` печатает признак завершения.

```
def process_arg(value):
    """Обрабатывает аргумент в зависимости от его типа."""
    try:
        if isinstance(value, list):
            neg_sum = sum(x for x in value if isinstance(x, (int, float)) and
x < 0)
            unique = list(dict.fromkeys(value)) # без повторов, порядок
сохраняется
            value[:] = unique # изменяем исходный список
            print(f"Список. Сумма отрицательных: {neg_sum}")
            print(f"Список без повторов: {value}")
            return neg_sum, value

        elif isinstance(value, set):
            words_count = sum(1 for item in value if isinstance(item, str)
and item.strip())
            print(f"Множество. Количество слов: {words_count}")
            return words_count
```

```

elif isinstance(value, bool):
    # bool – подкласс int, обрабатываем отдельно
    raise TypeError("Булево значение не поддерживается")

elif isinstance(value, (int, float)):
    n = int(value)
    if n < 2:
        is_prime = False
    else:
        is_prime = all(n % i != 0 for i in range(2, int(n ** 0.5) +
1))

    print(f"Число {n}: {'простое' if is_prime else 'не простое'}")
    return is_prime

elif isinstance(value, str):
    vowels_set = set("aeёиоуыэюяaeiouy")
    consonants_set = set("бвгджзйклмнпрстфхцчшщбсcdfghjklmnpqrstvwxyz")
    vowels = consonants = 0
    for ch in value.lower():
        if ch in vowels_set:
            vowels += 1
        elif ch in consonants_set:
            consonants += 1
    words = value.split()
    longest = max(words, key=len) if words else ""
    print(f"Строка. Гласных: {vowels}, согласных: {consonants}")
    print(f"Самое длинное слово: '{longest}'")
    return vowels, consonants, longest

else:
    raise TypeError(f"Неподдерживаемый тип: {type(value).__name__}")

except Exception as e:
    print(f"Ошибка при обработке: {e}")
    return None
finally:
    print("Обработка аргумента завершена.\n")

# --- Проверка со всеми случаями ---
print("=== Список ===")
lst = [1, -2, 3, -4, 1, -2, 5, 3]
process_arg(lst)

print("=== Множество ===")
process_arg({"яблоко", "груша", "киви", "банан"})

print("=== Число (простое) ===")
process_arg(17)

print("=== Число (не простое) ===")
process_arg(18)

print("=== Строка ===")
process_arg("Программирование на языке Python интересно")

```

Сумма отрицательных элементов списка равна минус 8, после удаления повторов остались 1, -2, 3, -4 и 5. Множество из четырёх строк дало четыре слова. Число 17 признано простым, число 18 – составным. В контрольной

строке 17 гласных и 21 согласная, самое длинное слово – «Программирование». Проверка всех случаев показана на рисунке 4.9.

```
=== Список ===
Список. Сумма отрицательных: -8
Список без повторов: [1, -2, 3, -4, 5]
Обработка аргумента завершена.

=== Множество ===
Множество. Количество слов: 4
Обработка аргумента завершена.

=== Число (простое) ===
Число 17: простое
Обработка аргумента завершена.

=== Число (не простое) ===
Число 18: не простое
Обработка аргумента завершена.

=== Строка ===
Строка. Гласных: 17, согласных: 21
Самое длинное слово: 'Программирование'
Обработка аргумента завершена.
```

Рисунок 4.9 – Обработка списка, множества, числа и строки

4.2.3 Знаки элементов матрицы

Сначала проверяют, что в каждой строке есть хотя бы один положительный элемент. Только при выполнении условия каждый элемент заменяют противоположным по знаку. Пустую или неверную структуру матрицы отклоняют в эксепт.

```
def invert_matrix_if_all_rows_positive(matrix):
    """Если в каждой строке есть положительный элемент — меняет знаки всех
    элементов."""
    try:
        if not matrix or not all(isinstance(row, (list, tuple)) for row in
matrix):
            raise ValueError("Матрица должна быть непустым списком строк")

        all_rows_ok = all(any(x > 0 for x in row) for row in matrix)

        if all_rows_ok:
            for i in range(len(matrix)):
                for j in range(len(matrix[i])):
                    matrix[i][j] = -matrix[i][j]
```

```

        print("Во всех строках есть положительный элемент. Знаки
изменены.")
    else:
        print("Не во всех строках есть положительный элемент. Матрица без
изменений.")

    return matrix

except Exception as e:
    print(f"Ошибка: {e}")
    return None
finally:
    print("Проверка матрицы завершена.")

matrix1 = [
    [1, -2, 0],
    [-5, 3, -1],
    [0, 0, 4],
]
print("Исходная матрица 1:")
for row in matrix1:
    print(row)

result1 = invert_matrix_if_all_rows_positive(matrix1)
print("Результат:")
for row in result1:
    print(row)

print()

matrix2 = [
    [1, 2],
    [-3, -4], # нет положительных
]
print("Исходная матрица 2:")
for row in matrix2:
    print(row)

result2 = invert_matrix_if_all_rows_positive(matrix2)
print("Результат:")
for row in result2:
    print(row)

```

В первой матрице условие выполнено, знаки изменены. Во второй матрице вторая строка положительных элементов не содержит, матрица осталась прежней. Оба случая показаны на рисунке 4.10.

```
Исходная матрица 1:
[1, -2, 0]
[-5, 3, -1]
[0, 0, 4]
Во всех строках есть положительный элемент. Знаки изменены.
Проверка матрицы завершена.
Результат:
[-1, 2, 0]
[5, -3, 1]
[0, 0, -4]

Исходная матрица 2:
[1, 2]
[-3, -4]
Не во всех строках есть положительный элемент. Матрица без изменений.
Проверка матрицы завершена.
Результат:
[1, 2]
[-3, -4]
```

Рисунок 4.10 – Проверка строк матрицы и смена знаков

4.2.4 Демонстрация try/except/finally

Делимое и делитель читают как вещественные числа. `ZeroDivisionError` и `ValueError` обрабатывают разными ветвями. Сообщение `finally` печатают при любом исходе.

```
def demo_try_except_finally():
    """Демонстрация try / except / finally: деление двух чисел."""
    print("=== Демонстрация try / except / finally ===\n")

    try:
        a = float(input("Введите делимое: "))
        b = float(input("Введите делитель: "))
        result = a / b
        print(f"Результат деления: {result}")
    except ZeroDivisionError:
        print("Ошибка: деление на ноль!")
    except ValueError:
        print("Ошибка: введено не число!")
    except Exception as e:
        print(f"Непредвиденная ошибка: {e}")
    finally:
        print("Блок finally: выполняется всегда (успех или ошибка).")

demo_try_except_finally()
```

Деление 15 на 3 дало 5, результат приведён на рисунке 4.11. Деление на ноль перехвачено ветвью `ZeroDivisionError`, как показано на рисунке 4.12. Ввод текста «abc» перехвачен ветвью `ValueError`, как показано на рисунке 4.13. Во всех трёх запусках блок `finally` выполнен, программа не остановилась.

```
=== Демонстрация try / except / finally ===  
  
Введите делимое: 15  
Введите делитель: 3  
Результат деления: 5.0  
Блок finally: выполняется всегда (успех или ошибка).
```

Рисунок 4.11 – Успешное деление

```
=== Демонстрация try / except / finally ===  
  
Введите делимое: 15  
Введите делитель: 0  
Ошибка: деление на ноль!  
Блок finally: выполняется всегда (успех или ошибка).
```

Рисунок 4.12 – Деление на ноль

```
=== Демонстрация try / except / finally ===  
  
Введите делимое: abc  
Ошибка: введено не число!  
Блок finally: выполняется всегда (успех или ошибка).
```

Рисунок 4.13 – Нечисловой ввод

4.3 Часть 3. Классы

4.3.1 Класс Example

Статические поля `static_x` и `static_y` записаны в теле класса. Динамические поля `dyn1` и `dyn2` создаются в инициализаторе. Глобальные `a` и `b` доступны второму методу. Третий метод возводит `dyn1` в степень `dyn2`.

```

a = 10 # глобальная переменная
b = 20 # глобальная переменная

class Example:
    # две статические (классовые) переменные
    static_x = 5
    static_y = 7

    def __init__(self, dyn1, dyn2):
        # две динамические (экземплярные) переменные
        self.dyn1 = dyn1
        self.dyn2 = dyn2

    def method1(self):
        """Создаёт переменную и выводит её."""
        local_var = "локальная переменная метода 1"
        print(local_var)

    def method2(self):
        """Возвращает сумму двух глобальных переменных."""
        return a + b

    def method3(self):
        """Возвращает dyn1 ** dyn2."""
        return self.dyn1 ** self.dyn2

obj = Example(2, 8)

print("Метод 1:")
obj.method1()

print("Метод 2 (сумма глобальных a и b):", obj.method2())
print("Метод 3 (dyn1 ** dyn2):", obj.method3())
print("Переменная a:", a)

```

Первый метод напечатал локальную переменную. Сумма глобальных *a* и *b* равна 30. Возведение 2 в степень 8 дало 256. Переменная *a* равна 10. Результат приведён на рисунке 4.14.

Метод 1: локальная переменная метода 1 Метод 2 (сумма глобальных <i>a</i> и <i>b</i>): 30 Метод 3 (<i>dyn1</i> ** <i>dyn2</i>): 256 Переменная <i>a</i>: 10
--

Рисунок 4.14 – Методы класса *Example* и переменная *a*

4.3.2 Класс Country

Объект хранит название, столицу, площадь и численность населения. Отбор по площади и по населению вынесен в отдельные функции: в список попадает страна, у которой показатель не меньше заданного порога.

```
class Country:
    def __init__(self, name, capital, area, population):
        self.name = name
        self.capital = capital
        self.area = area          # км²
        self.population = population # чел.

    def __str__(self):
        return (
            f"{self.name}: столица – {self.capital}, "
            f"площадь – {self.area} км², "
            f"население – {self.population} чел."
        )

countries = [
    Country("Россия", "Москва", 17_098_246, 146_000_000),
    Country("Франция", "Париж", 643_801, 67_000_000),
    Country("Япония", "Токио", 377_975, 125_000_000),
    Country("Беларусь", "Минск", 207_600, 9_200_000),
    Country("Казахстан", "Астана", 2_724_900, 19_000_000),
]

def by_area(country_list, min_area):
    """Страны с площадью >= заданной."""
    return [c for c in country_list if c.area >= min_area]

def by_population(country_list, min_population):
    """Страны с населением >= заданного."""
    return [c for c in country_list if c.population >= min_population]

area_limit = 500_000
pop_limit = 50_000_000

print(f"a) Страны с площадью >= {area_limit} км²:")
for c in by_area(countries, area_limit):
    print(" ", c)

print(f"\nb) Страны с населением >= {pop_limit} чел.:")
for c in by_population(countries, pop_limit):
    print(" ", c)
```

При пороге площади 500 000 км² отобраны Россия, Франция и Казахстан. При пороге населения 50 000 000 человек отобраны Россия, Франция и Япония. Списки приведены на рисунке 4.15.

- а) Страны с площадью ≥ 500000 км²:
- Россия: столица – Москва, площадь – 17098246 км², население – 146000000 чел.
Франция: столица – Париж, площадь – 643801 км², население – 67000000 чел.
Казахстан: столица – Астана, площадь – 2724900 км², население – 19000000 чел.
- б) Страны с населением ≥ 50000000 чел.:
- Россия: столица – Москва, площадь – 17098246 км², население – 146000000 чел.
Франция: столица – Париж, площадь – 643801 км², население – 67000000 чел.
Япония: столица – Токио, площадь – 377975 км², население – 125000000 чел.

Рисунок 4.15 – Страны по площади и по численности населения

4.3.3 Канцелярские принадлежности

Базовый метод `draw` печатает сообщение о запуске отрисовки. `Pen`, `Pencil` и `Handle` наследуют `title` и заменяют `draw` собственным текстом. Вызов идёт по одному имени, а выполняется версия того класса, к которому принадлежит объект.

```
class Stationery:
    def __init__(self, title):
        self.title = title

    def draw(self):
        print("Запуск отрисовки")

class Pen(Stationery):
    def draw(self):
        print(f"Ручка «{self.title}»: рисуем чернилами")

class Pencil(Stationery):
    def draw(self):
        print(f"Карандаш «{self.title}»: рисуем графитом")

class Handle(Stationery):
    def draw(self):
        print(f"Маркер «{self.title}»: рисуем ярким цветом")

base = Stationery("Базовый инструмент")
pen = Pen("Parker")
pencil = Pencil("Koh-i-Noor")
handle = Handle("Edding")

print("Stationery:")
base.draw()

print("\nPen:")
pen.draw()
```

```
print("\nPencil:")
pencil.draw()

print("\nHandle:")
handle.draw()
```

Экземпляр Stationery вывел общее сообщение. Ручка, карандаш и маркер вывели собственные сообщения. Проверка показана на рисунке 4.16.

```
Stationery:
Запуск отрисовки

Pen:
Ручка «Parker»: рисуем чернилами

Pencil:
Карандаш «Koh-i-Noor»: рисуем графитом

Handle:
Маркер «Edding»: рисуем ярким цветом
```

Рисунок 4.16 – Метод draw базового класса и трёх наследников

4.3.4 Класс Car

В качестве собственного примера взят учёт автомобилей. Метод экземпляра info возвращает марку, модель и расход. Метод cost_for_distance считает стоимость поездки. Статический метод is_economical сравнивает расход с порогом 7 л на 100 км и не обращается к объекту. Метод класса from_string создаёт автомобиль из строки «марка; модель; расход», метод get_fleet_count возвращает число созданных объектов.

```
class Car:
    fleet_count = 0 # сколько машин создано

    def __init__(self, brand, model, fuel_consumption):
        self.brand = brand
        self.model = model
        self.fuel_consumption = fuel_consumption # л / 100 км
        Car.fleet_count += 1

    # --- метод экземпляра ---
    def info(self):
        return f"{self.brand} {self.model}, расход {self.fuel_consumption} л/100 км"

    def cost_for_distance(self, km, price_per_liter):
        """Стоимость поездки на заданное расстояние."""
        liters = self.fuel_consumption * km / 100
```

```

        return liters * price_per_liter

# --- статический метод ---
@staticmethod
def is_economical(consumption):
    """Экономичный ли расход (без доступа к self / cls)."""
    return consumption <= 7.0

# --- метод класса ---
@classmethod
def from_string(cls, data):
    """Создаёт объект из строки 'Brand;Model;Consumption'."""
    brand, model, consumption = data.split(";")
    return cls(brand.strip(), model.strip(), float(consumption))

@classmethod
def get_fleet_count(cls):
    return cls.fleet_count

car1 = Car("Toyota", "Corolla", 6.5)
car2 = Car.from_string("BMW; X5; 11.2")

print("Авто 1:", car1.info())
print("Авто 2:", car2.info())
print(f"Поездка 200 км на {car1.model}: {car1.cost_for_distance(200, 55):.2f} руб.")
print("Corolla экономичная?", Car.is_economical(car1.fuel_consumption))
print("X5 экономичный?", Car.is_economical(car2.fuel_consumption))
print("Всего автомобилей в парке:", Car.get_fleet_count())

```

Поездка на 200 км для Corolla при цене 55 руб. за литр стоит 715 руб. Расход 6,5 л на 100 км признан экономичным, расход 11,2 л на 100 км – нет. Второй автомобиль создан разбором строки. Счётчик парка равен 2. Результат приведён на рисунке 4.17.

```

Авто 1: Toyota Corolla, расход 6.5 л/100 км
Авто 2: BMW X5, расход 11.2 л/100 км
Поездка 200 км на Corolla: 715.00 руб.
Corolla экономичная? True
X5 экономичный? False
Всего автомобилей в парке: 2

```

Рисунок 4.17 – Методы экземпляра, статический метод и метод класса Car

5 ВЫВОДЫ

По варианту 17 составлены и проверены программы трёх частей лабораторной работы.

В первой части получены простые числа заданного диапазона, разделены гласные и согласные, посчитаны пары равных элементов и частоты цифр. Меню магазина автозапчастей показывает описание, цену и остаток, а при покупке уменьшает остаток и считает сумму. Для кортежа найден индекс первого отрицательного элемента.

Во второй части функции различают тип аргумента. Неверный ввод не останавливает программу: ошибка перехватывается, а блок `finally` выполняется при любом исходе. Знаки матрицы меняются только тогда, когда положительный элемент есть в каждой строке.

В третьей части разделены общие и объектные атрибуты, метод родителя заменён в классах-наследниках, а на классе автомобиля показаны метод экземпляра, статический метод и метод класса.

Поставленная цель достигнута: перечисленные средства языка применены к индивидуальному заданию и проверены контрольными запусками.